

Enterprise Databricks Migration Factory

Comprehensive Architecture, Release Status & Release 4 Implementation Plan

1. Executive Summary & Objective

The application secures, automates, and governs the end-to-end migration from Microsoft SQL Server to Databricks Unity Catalog. The platform operates on a strict metadata-first architecture: AI models act as planners and code translators, but **never execute arbitrary SQL** or bypass governance controls. All data movement, environment creation, and artifact deployments are executed deterministically through controlled backend services with human approval gates.

2. Program Delivery Roadmap & Current Milestone Status

Release	Focus Area & Core Capabilities	Status
Release 1	Governed Environment Provisioning: Project-scoped Databricks configuration, secret references, preflight validation matrix, and idempotent DEV catalog/schema creation (migration_dev: bronze, silver, gold).	COMPLETED
Release 2	DEV Bronze Ingestion: Discovered SQL Server table streaming to <catalog>_dev.bronze via FULL_LOAD (with atomic staging Delta tables) & APPEND, type normalization, and row-count validation.	COMPLETED
Release 3	Governed Prompt Migration Orchestrator: Natural-language intent parser ('Migrate MigrationDemo to DEV Databricks'), prerequisite validation, impact assessment preview, approval gate, and end-to-end execution runner.	COMPLETED
Release 4	Automated Silver/Gold Modeling & Governed AI Remediation: Automatic Silver cleansing, Gold star-schema marts, static SQL validation, and bounded idempotent AI repair loop with human review gates.	NEXT MILESTONE
Release 5	Reconciliation & Controlled Multi-Tier Promotion: Sequential promotion gates (DEV → TEST → UAT → PROD) with identical artifact version integrity, reconciliation diffs, and rollback tracking.	SCHEDULED

3. Release 3 Highlights (Delivered)

Release 3 connects prompt-based natural language interaction with the governed backend migration APIs:

- **Intent Parsing & Guardrails:** Interprets prompt statements, identifies source database and target tier, and returns NEEDS_USER_INPUT if credentials or DEV catalogs are unprovisioned.
- **Pre-Execution Impact Assessment:** Previews table counts, volume estimates, risk level, and destination table mappings before execution.
- **Mandatory Approval Gate:** Prompts initiate and plan, but no data moves until the operator reviews and clicks *Approve & Execute*.
- **Test Verification:** 169 backend unit and integration tests passing with 0 regressions.

4. Release 4 Implementation Plan: Automated Medallion & AI Remediation

Release 4 automates the analytical layer transformation from raw Bronze tables into curated Silver views and dimensional Gold marts, backed by deterministic static validation and a bounded AI repair loop.

A. Automated Silver Layer Engine

- **Data Cleaning & Normalization:** Automatically normalizes column names to snake_case, handles nulls, trims strings, and casts timestamps to UTC.
- **Deduplication:** Applies windowed deduplication based on discovered primary/business keys (ROW_NUMBER() OVER (PARTITION BY pk ORDER BY updated_at DESC)).
- **Explicit Lineage:** Stores lineage links from SQL Server source object → Bronze Delta table → Silver cleansed entity.

B. Automated Gold Layer Modeling

- **Star-Schema Generation:** Automatically creates dimension views (dim_customer, dim_product) and fact views (fact_orders).
- **Surrogate & Business Keys:** Generates deterministic surrogate keys using sha2(concat_ws('||', ...)) for cross-system consistency.
- **Conformed Dimensions & KPIs:** Defines reusable business grain metrics and aggregation views.

C. Deterministic Static Validation Engine

- **Databricks SQL Syntax Verification:** Pre-validates DDL/DML statements against Databricks Delta SQL grammar before execution.
- **Column & Alias Integrity:** Inspects all referenced columns against underlying schema tables; prevents missing-column deployment blockers.
- **Cyclic Dependency Detection:** Uses topological graph ordering to ensure tables and views deploy in exact dependency sequence.

D. Governed AI Remediation Loop

- **Bounded Context:** Sends only the failing SQL query, relevant table column definitions, and the specific compiler error to Gemini—no raw customer data or credentials.
- **Idempotency & Caching:** Employs idempotency keys and caches verified remediation templates to prevent redundant token consumption.
- **Transient Fault Backoff:** Automatically retries temporary HTTP 429 and 5xx errors with exponential backoff.
- **Deterministic Re-Validation:** AI-generated fixes must pass static syntax checks before being presented for review.
- **Immutable Versioning:** Accepted corrections increment artifact versions (e.g. v1 → v2).
- **Human Approval Gate:** AI is strictly barred from self-approving or auto-deploying its own repairs.

5. Release 5 Roadmap: Controlled Promotion (TEST → UAT → PROD)

Release 5 completes the promotion pipeline by enforcing governed stage progression:

Promotion Stage	Governance Requirement & Controls
DEV → TEST	Automated reconciliation check, unit test validation pass, and standard migration architect sign-off.

TEST → UAT	Data quality test suite pass, schema stability drift check, and QA team approval sign-off.
UAT → PROD	Strict dual-authorization gate (Release Manager + Lead Architect), dry-run preflight, and rollback checkpoint registration.
Version Integrity	Guarantees the exact same artifact version validated in DEV is promoted across all environments. Modifying code in PROD is blocked.

6. Security & Governance Guarantees

Zero Secret Exposure: Credentials remain in the runtime environment; secret references only (e.g. CLIENT_A_DATABRICKS_TOKEN) are stored. **Non-Destructive Operations:** All provisioning uses IF NOT EXISTS; DROP CATALOG, DROP SCHEMA, and TRUNCATE are hard-blocked by the execution engine. **Auditability:** Every planning, preflight, approval, deployment, and remediation action is recorded as an immutable CanonicalRecord in the metadata database.